

Actividad 2

Ing. Isaias Garcia Zanella

25 de febrero de 2026

1. Introducción

En esta actividad se realizará una investigación más profunda sobre las mediciones de distancia que existen y también se realizará una práctica con Python para la imputación de datos en bases de datos (B.D.) con datos faltantes, además de la identificación de datos atípicos y la codificación y normalización de datos para su posterior uso, ya sea en modelos de Machine Learning o en interpretaciones gráficas.

2. Marco Teórico

2.1. Medidas de Distancia

En muchos algoritmos de IA, el cálculo de la distancia es un elemento crucial para calcular la similaridad entre conjuntos de vectores. [1]

Esto es importante para algoritmos de clustering, clasificación, reducción de dimensionalidad, entre otros. Existen varias medidas de distancia, cada una con sus propias características y aplicaciones. A continuación, se describen algunas de las medidas de distancia más comunes:

2.1.1. Distancia Euclidiana

La distancia euclidiana está basada en la distancia bidimensional entre dos vectores; es decir, la distancia entre dos puntos como si se dibujara una línea con una regla de un punto al otro. Esta es una de las distancias que más se utiliza en algoritmos de aprendizaje automático.

Específicamente, si se tienen dos puntos (x_1, y_1) y (x_2, y_2) , la distancia euclidiana puede ser calculada usando la fórmula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (1)$$

[1]

2.1.2. Distancia de Manhattan

La distancia de Manhattan es también llamada distancia del taxi (o «Taxicab»). Se puede entender mejor esta distancia como si alguien estuviera manejando por una ciudad. Para calcular la distancia de Manhattan entre dos puntos, se suman las distancias absolutas como se muestra en la siguiente ecuación:

$$D = \sum_{i=1}^n |x_i - y_i| \quad (2)$$

[1]

2.1.3. Distancia de Chebyshev

La distancia de Chebyshev también es muy utilizada en algoritmos de aprendizaje automático. Se conoce comúnmente como distancia del ajedrez. En el ajedrez, cada pieza se mueve de manera diferente en el tablero. En este sentido, la distancia de Chebyshev se puede interpretar como el número de movimientos en los que el rey puede llegar desde su posición inicial a cierta posición en el tablero.

La representación matemática de la distancia de Chebyshev se muestra en la siguiente ecuación:

$$D_{Cheb} = \max(|x_2 - x_1|, |y_2 - y_1|) \quad (3)$$

[1]

2.1.4. Distancia de Hamming

La distancia de Hamming mide cuántos elementos son diferentes entre dos secuencias del mismo tamaño. Es como contar los “errores” o diferencias entre dos cadenas de datos.

Matemáticamente se expresa por medio de la siguiente ecuación:

$$d_{p,q} = \sum_{i=1}^n \delta(p_i, q_i) \quad (4)$$

Donde:

- p y q son dos cadenas o vectores del tamaño n .
- $\delta(p_i, q_i)$ es 1 si $p_i \neq q_i$, y 0 si $p_i = q_i$.

[1]

2.1.5. Distancia de Bhattacharyya

La distancia de Bhattacharyya mide la similitud entre dos distribuciones de probabilidad. Se utiliza principalmente para evaluar qué tan “parecidas” son dos conjuntos de datos representados como distribuciones.

Matemáticamente, se puede expresar como en la ecuación siguiente:

$$D_B(p, q) = -\ln \left(\sum_{i=1}^n \sqrt{p_i \cdot q_i} \right) \quad (5)$$

Donde: p_i y q_i son las probabilidades asociadas a la i -ésima categoría o intervalo en las distribuciones p y q .

La suma $\left(\sum_{i=1}^n \sqrt{p_i \cdot q_i} \right)$ se conoce como el coeficiente de Bhattacharyya. Si las distribuciones son idénticas, el coeficiente será 1 y la distancia es 0. Cuanto más diferentes sean, mayor será la distancia. [1]

2.1.6. Distancia de Jaccard

La **distancia de Jaccard** mide cuán diferentes son dos conjuntos al comparar los elementos que tienen en común frente a los que no. Es una métrica basada en el concepto de intersección y unión de conjuntos.

Matemáticamente, se puede expresar de la siguiente manera:

$$d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} \quad (6)$$

Donde:

- A y B son dos conjuntos.
- $|A \cap B|$ es el número de elementos que están en ambos conjuntos (intersección).
- $|A \cup B|$ es el número de elementos que están en al menos uno de los conjuntos (unión).

[1]

2.1.7. Distancia de Mahalanobis

La **distancia de Mahalanobis** es una métrica que mide la distancia entre un punto y un conjunto de datos, teniendo en cuenta la correlación y la variabilidad de las variables. A diferencia de la distancia euclidiana, no asume que todas las variables tienen la misma importancia o escala, sino que ajusta la medición según cómo están distribuidas las variables y cómo se relacionan entre sí.

La distancia de Mahalanobis entre un punto p y un conjunto de datos representado por el vector promedio μ es:

$$d(p, q) = \sqrt{(p - \mu)^T S^{-1} (p - \mu)} \quad (7)$$

Donde:

- p : es el punto en cuestión (un vector de características).
- μ : es el vector promedio (media) de los datos.
- S : es la matriz de covarianza de los datos.

- S^{-1} : es la inversa de la matriz de covarianza.

Esta medida de distancia tiene las siguientes aplicaciones:

- **Detección de anomalías:** Detectar valores atípicos en conjuntos de datos.
- **Clasificación estadística:** Clasificar puntos en función de su cercanía a un grupo con una distribución conocida.
- **Análisis multivariable:** Comparar observaciones en datasets donde las variables tienen diferentes escalas y están correlacionadas.

Esta medida de distancia también tiene las siguientes limitaciones:

- **Requiere matriz de covarianza:** Si el número de observaciones es menor que el número de variables, la matriz de covarianza puede ser singular y no invertible.
- **Supone distribución normal:** La distancia de Mahalanobis funciona mejor si las variables siguen una distribución normal multivariada.
- **Sensibilidad a valores atípicos:** Los outliers pueden distorsionar la matriz de covarianza, afectando la medición de distancias.
- **Costo computacional:** Calcular la inversa de la matriz de covarianza puede ser difícil en datasets grandes o de alta dimensión.

[1]

2.1.8. Distancia de Bray-Curtis

La **distancia de Bray-Curtis** es una medida utilizada para comparar la diferencia relativa entre dos conjuntos de datos, especialmente cuando los datos representan cantidades (como conteos o proporciones). Esta métrica es muy popular en estudios ecológicos y análisis de comunidades, donde se quiere medir la disimilitud entre dos muestras basándose en sus composiciones.

La distancia de Bray-Curtis entre dos puntos p y q en un espacio de n dimensiones es:

$$d(p, q) = \frac{\sum_{i=1}^n |p_i - q_i|}{\sum_{i=1}^n (p_i + q_i)} \quad (8)$$

Donde:

- p_i y q_i son los valores para la i -ésima variable.
 - El numerador es la suma de las diferencias absolutas entre los valores de cada dimensión.
 - El denominador es la suma total de los valores combinados.
- El valor de $d(p, q)$ está entre 0 y 1:
- **0** significa que los dos conjuntos son completamente iguales.
 - **1** indica que no comparten ningún elemento en común.

[1]

2.1.9. Distancia de Canberra

La distancia de Canberra es una medida de disimilitud que compara dos puntos en un espacio multidimensional, poniendo más peso a las diferencias en dimensiones pequeñas. Es especialmente útil cuando se quiere resaltar diferencias relativas en valores cercanos a cero.

La distancia de Canberra entre dos puntos p y q con n dimensiones es:

$$d(p, q) = \sum_{i=1}^n \frac{|p_i - q_i|}{|p_i| + |q_i|} \quad (9)$$

Donde:

- p_i y q_i son los valores en la dimensión i .
- El numerador mide la diferencia absoluta.
- El denominador normaliza esta diferencia en función de la suma de los valores.

[1]

2.2. Datos Faltantes

En el análisis de datos, los datos faltantes (missing data) son valores que no se encuentran registrados en una o más variables del conjunto de datos. La presencia de datos faltantes puede afectar significativamente el desempeño de los modelos estadísticos y de aprendizaje automático, generando sesgos o reduciendo la precisión de las predicciones [5].

Los datos faltantes se clasifican generalmente en tres categorías [5]:

- **MCAR (Missing Completely At Random)**: Los datos faltan completamente al azar y su ausencia no depende de ninguna variable observada o no observada.
- **MAR (Missing At Random)**: La ausencia depende de variables observadas, pero no del valor faltante en sí.
- **MNAR (Missing Not At Random)**: La ausencia depende del propio valor faltante.

El tratamiento adecuado de los datos faltantes es fundamental para garantizar la validez del análisis posterior [3].

2.3. Datos Atípicos (IQR)

Los datos atípicos (outliers) son observaciones que se alejan significativamente del comportamiento general del conjunto de datos. Estos pueden surgir debido a errores de medición, errores de registro o variabilidad natural del fenómeno estudiado [6].

Una técnica común para detectar valores atípicos es el método del rango intercuartílico (IQR, Interquartile Range), propuesto en el contexto del análisis exploratorio de datos [6]. El IQR se define como:

$$IQR = Q_3 - Q_1 \quad (10)$$

donde Q_1 es el primer cuartil (percentil 25) y Q_3 es el tercer cuartil (percentil 75).

Un valor se considera atípico si cumple:

$$x < Q_1 - 1,5(IQR) \quad \text{o} \quad x > Q_3 + 1,5(IQR) \quad (11)$$

Este método es ampliamente utilizado debido a su simplicidad y robustez frente a distribuciones no normales [2].

2.4. Imputación

La imputación es el proceso mediante el cual se reemplazan los valores faltantes por estimaciones plausibles, con el objetivo de preservar la mayor cantidad de información posible sin introducir sesgos significativos [5].

Existen diferentes métodos de imputación [3, 4]:

- **Imputación por media, mediana o moda:** Sustituye los valores faltantes por estadísticas descriptivas.
- **Imputación por regresión:** Utiliza modelos estadísticos para estimar valores faltantes.
- **K-Nearest Neighbors (KNN):** Imputa valores basándose en observaciones similares.
- **Imputación múltiple:** Genera múltiples conjuntos de datos imputados para reducir la incertidumbre.

La elección del método depende del tipo de variable y del mecanismo de ausencia de los datos [5].

2.5. Codificación

La codificación es el proceso de transformar variables categóricas en representaciones numéricas para que puedan ser utilizadas por algoritmos de aprendizaje automático [2].

Algunos métodos comunes incluyen [4]:

- **Label Encoding:** Asigna un número entero a cada categoría.
- **One-Hot Encoding:** Crea una variable binaria por cada categoría.
- **Target Encoding:** Codifica las categorías según la variable objetivo.

La selección del método adecuado depende del tipo de modelo y de la naturaleza de los datos [3].

2.6. Normalización

La normalización es una técnica de preprocesamiento que consiste en escalar los datos numéricos para que se encuentren dentro de un rango específico o tengan propiedades estadísticas determinadas [2].

Algunos métodos comunes son [4]:

- **Min-Max Scaling:**

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (12)$$

- **Estandarización (Z-score):**

$$x' = \frac{x - \mu}{\sigma} \quad (13)$$

La normalización es especialmente importante en algoritmos basados en distancia, como KNN, SVM o métodos de clustering, ya que evita que variables con mayor escala dominen el cálculo [3].

3. Materiales y Métodos

3.1. Materiales

Para el desarrollo de la presente actividad se utilizó el siguiente equipo de cómputo y herramientas de software:

- **Equipo:** Laptop HP Victus.
- **Procesador:** AMD Ryzen 7000 Series.
- **Tarjeta gráfica:** NVIDIA GeForce RTX 4050.
- **Lenguaje de programación:** Python.
- **Entorno de desarrollo:** Visual Studio Code (VS Code).
- **Bibliotecas utilizadas:** Pandas (para manejo de datos y detección de valores nulos).

El equipo permitió ejecutar los algoritmos de análisis de datos, detección de valores atípicos, imputación, codificación y normalización de manera eficiente. Visual Studio Code se empleó como entorno de desarrollo integrado (IDE) para la escritura, ejecución y depuración del código, facilitando la organización del proyecto y la gestión de librerías.

3.2. Métodos

El procesamiento de datos se realizó mediante funciones programadas en Python, utilizando la biblioteca Pandas para el manejo de valores nulos.

3.2.1. Detección de Datos Faltantes

Se implementó la siguiente función para calcular el porcentaje de valores faltantes:

Listing 1: Funcion para calcular porcentaje de datos faltantes

```
1 def get_empty_Data(data_list):
2     empty_count = sum(1 for x in data_list if pd.isna(x))
3     porcentaje = (empty_count / len(data_list)) * 100
4     return porcentaje
```

La función cuenta los valores nulos y calcula su proporción respecto al total de datos.

3.2.2. Detección de Valores Atípicos (IQR)

Para la detección de valores atípicos se implementó el siguiente algoritmo basado en el rango intercuartílico:

Listing 2: Deteccion de valores atipicos mediante IQR

```
1 def calcular_estadisticas_outliers(data_list):
2     if not data_list:
3         return None
4
5     data_ordenada = sorted(data_list)
6
7     def get_percentile(data, percentile):
8         k = (len(data) - 1) * percentile
9         f = int(k)
10        c = k - f
11        if f + 1 < len(data):
12            return data[f] + (data[f+1] - data[f]) * c
13        else:
14            return data[f]
15
16        q1 = get_percentile(data_ordenada, 0.25)
17        q3 = get_percentile(data_ordenada, 0.75)
18        iqr = q3 - q1
19
20        limite_inferior = q1 - 1.5 * iqr
21        limite_superior = q3 + 1.5 * iqr
22
23        outliers = [x for x in data_list
24                    if x < limite_inferior or x > limite_superior]
25
26        return {
27            "q1": q1,
28            "q3": q3,
29            "iqr": iqr,
30            "limite_inf": limite_inferior,
31            "limite_sup": limite_superior,
32            "outliers": outliers
33        }
```

El método calcula los cuartiles, el IQR y determina los límites inferior y superior para identificar valores atípicos.

3.2.3. Métodos de Imputación

Listing 3: Imputacion por media

```
1 def imputacion_media(data_list):
2     media = get_mean(data_list)
3     data_imputada = [media if pd.isna(x) else x
4                     for x in data_list]
5     cambios = []
6     for i, x in enumerate(data_list):
7         if pd.isna(x):
8             cambios.append({"indice": i,
9                             "valor_nuevo": media})
10
11     return data_imputada, cambios
```

Listing 4: Imputacion por vecinos mas cercanos (KNN)

```
1 def imputacion_knn(data_list, k=3):
2     data_imputada = []
3     cambios = []
4
5     for i, x in enumerate(data_list):
6         if pd.isna(x):
7             vecinos = []
8
9             for j in range(i-1, max(i-k-1, -1), -1):
10                 if not pd.isna(data_list[j]):
11                     vecinos.append(data_list[j])
12                 if len(vecinos) >= k:
13                     break
14
15             for j in range(i+1, min(i+k+1, len(data_list))):
16                 if not pd.isna(data_list[j]):
17                     vecinos.append(data_list[j])
18                 if len(vecinos) >= k:
19                     break
20
21             if vecinos:
22                 nuevo_valor = sum(vecinos) / len(vecinos)
23                 data_imputada.append(nuevo_valor)
24                 cambios.append({
25                     "indice": i,
26                     "valor_nuevo": nuevo_valor,
27                     "vecinos_usados": vecinos.copy()
28                 })
29             else:
30                 data_imputada.append(x)
31         else:
32             data_imputada.append(x)
```

```

33
34     return data_imputada, cambios

```

Listing 5: Codificación One-Hot

```

1 def codificacion_one_hot(data_list):
2     categorias = sorted(set(x for x in data_list
3                             if str(x).lower() != 'nan'))
4     codificacion = {cat: [1 if x == cat else 0
5                           for x in data_list]
6                     for cat in categorias}
7     return codificacion

```

Listing 6: Normalización Min-Max

```

1 def normalizacion_min_max(data_list):
2     min_val = get_min_value(data_list)
3     max_val = get_max_value(data_list)
4
5     if max_val == min_val:
6         return [0.5 for _ in data_list]
7
8     return [(x - min_val) / (max_val - min_val)
9             if not pd.isna(x) else x
10            for x in data_list]

```

El procesamiento de datos se realizó mediante funciones programadas en Python. Se implementaron algoritmos para detectar datos faltantes, identificar valores atípicos, aplicar métodos de imputación, realizar codificación de variables categóricas y normalizar los datos numéricos.

4. Resultados y Discusión

Para la observación y análisis de los datos se utilizó la base de datos disponible en la plataforma Kaggle titulada “*Diabetes Dataset with Missing Value for Practice*”.

Este conjunto de datos fue seleccionado debido a que contiene valores faltantes de manera intencional, lo que permite aplicar y evaluar los métodos de imputación, detección de valores atípicos, codificación y normalización desarrollados en la sección anterior.

El conjunto de datos puede consultarse en el siguiente enlace:

<https://www.kaggle.com/datasets/bhavyamotiyani/diabetes-dataset-with-missing-value-for-practice>

Para el análisis presentado en esta sección se trabajó específicamente con la columna **Skin_Fold**, ya que esta variable contiene valores faltantes y resulta adecuada para evaluar el impacto de los métodos de imputación y normalización.

4.1. Imputación de Datos mediante KNN

Para el tratamiento de valores faltantes en la variable **Skin_Fold** se empleó el método de los vecinos más cercanos (KNN), el cual sustituye cada valor ausente con base en los valores más próximos dentro del conjunto de datos. Este enfoque permite preservar la estructura

local y la variabilidad original de la variable, evitando concentraciones artificiales en un único valor.

Con el fin de evaluar el impacto de la imputación, se realizó una comparación de densidades utilizando estimación Kernel (KDE), contrastando la distribución de los valores originales con la distribución obtenida tras la imputación.

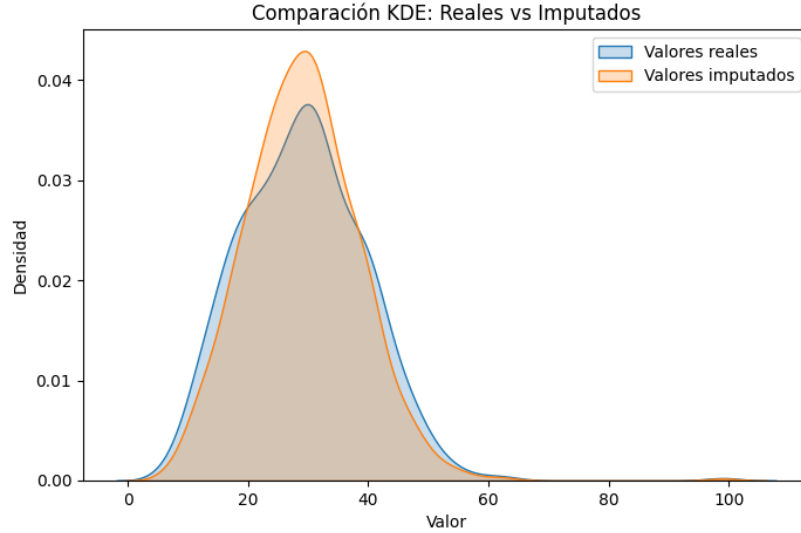


Figura 1: Comparación de densidad (KDE) entre valores originales e imputados mediante KNN para la variable `Skin_Fold`.

Como se observa en la Figura 1, ambas curvas presentan una superposición considerable, lo que indica que el método KNN preserva adecuadamente la forma general de la distribución. No se aprecia un desplazamiento significativo del pico principal ni alteraciones notables en la dispersión de los datos, lo que sugiere que la imputación no introduce sesgos estadísticos relevantes.

Por lo tanto, puede concluirse que el método KNN resultó apropiado para esta variable, manteniendo la coherencia estadística del conjunto de datos.

4.2. Codificación de Variables

Posteriormente, se aplicó el proceso de codificación correspondiente a las variables categóricas del conjunto de datos. La codificación transforma las categorías en representaciones numéricas sin alterar la frecuencia original de cada clase.

Se verificó que la transformación mantiene la consistencia de la información, asegurando que la estructura del conjunto de datos permanezca intacta para su posterior análisis.

4.3. Normalización Min-Max

Finalmente, se aplicó la normalización Min-Max a la variable `Skin_Fold` con el propósito de escalar los valores al intervalo $[0,1]$, utilizando la transformación:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Para validar la correcta implementación del método, se graficó la relación entre los valores originales y los valores normalizados.

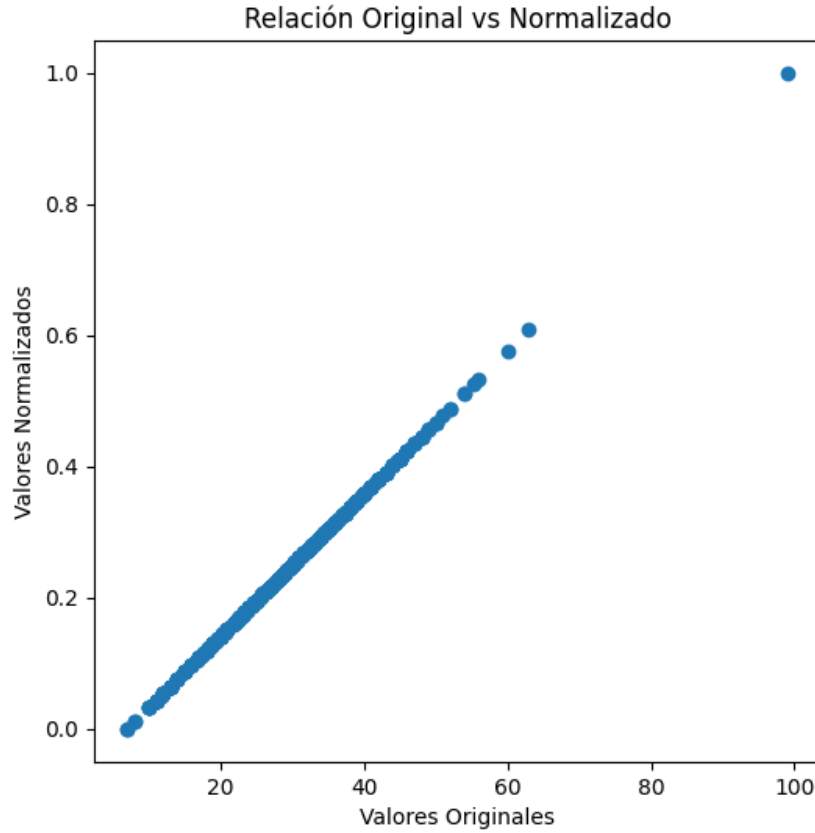


Figura 2: Relación entre valores originales y valores normalizados mediante Min-Max para la variable `Skin_Fold`.

En la Figura 2 se observa que los puntos siguen una tendencia lineal creciente, lo que confirma que la transformación aplicada es lineal. Esto implica que el orden relativo de los datos se conserva y que no se altera la proporción entre los valores originales.

Además, se verificó que el valor mínimo de la variable se transformó en 0 y el valor máximo en 1, cumpliendo con la propiedad fundamental del método Min-Max. En consecuencia, la normalización fue implementada correctamente y permite que la variable pueda ser utilizada en modelos que requieren escalamiento previo.

5. Conclusión

El presente trabajo permitió desarrollar e integrar de manera estructurada un proceso completo de preprocesamiento de datos, abarcando desde la gestión de valores faltantes hasta la normalización de variables, utilizando como caso de estudio el conjunto de datos “*Diabetes Dataset with Missing Value for Practice*” disponible en Kaggle.

En primer lugar, se analizaron y aplicaron métodos de imputación, destacando la imputación por media y el método de los vecinos más cercanos (KNN). A partir de la comparación mediante estimación de densidad Kernel (KDE), se observó que el método KNN preserva de mejor manera la forma de la distribución original, manteniendo la variabilidad y reduciendo la introducción de sesgos estadísticos. Esto evidencia la importancia de seleccionar técnicas de imputación que conserven la estructura interna de los datos.

Posteriormente, se realizó el proceso de codificación de variables categóricas, garantizando que la transformación mantuviera la coherencia y frecuencia original de las categorías. Esta etapa permitió adaptar el conjunto de datos para su posible utilización en modelos de aprendizaje automático sin pérdida de información relevante.

En la etapa final, se aplicó la normalización Min-Max, comprobando mediante representación gráfica que la transformación es lineal y conserva el orden relativo de los datos. La validación visual confirmó que el escalamiento fue implementado correctamente, asegurando que las variables se encuentren en un rango adecuado para algoritmos sensibles a la magnitud de los valores.

Bibliográficas

- [1] Dr. Marco Antonio Aceves Fernandez. *De 0 a 100 en Inteligencia Artificial. Todas las claves para comenzar tu viaje por la IA*. 2025. Cap. 2.7, págs. 54-102.
- [2] Jiawei Han, Micheline Kamber y Jian Pei. *Data Mining: Concepts and Techniques*. 3.^a ed. Morgan Kaufmann, 2011.
- [3] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning*. 2.^a ed. Springer, 2009.
- [4] Gareth James et al. *An Introduction to Statistical Learning*. 2.^a ed. Springer, 2021.
- [5] Roderick J.A. Little y Donald B. Rubin. *Statistical Analysis with Missing Data*. 3.^a ed. Wiley, 2019.
- [6] John W. Tukey. «Exploratory Data Analysis». En: *Addison-Wesley* (1977).